

Høyere-ordens funksjoner (kap 7)

- ▶ Funksjoner er første-classes innbyggere i FP
- ▶ Funksjoner kan være **inndata**, f.eks.:

```
filter filter (> 5) [1,3,5,7,9] = [7,9]  
filter (/=3) [1,2,3,4,5] = [1,2,4,5]
```

```
filter :: (a -> Bool) -> [a] -> [a]
```

```
all all (<10) [1,3,5,7,9] = True  
all (/=5) [1,2,3,4,5] = False
```

```
all :: (a -> Bool) -> [a] -> Bool
```

```
any any (<10) [1,3,5,7,9,10] = True  
any (/=5) [1,2,3,4,5] = True
```

```
any :: (a -> Bool) -> [a] -> Bool
```

Høyere-ordens funksjoner

- ▶ Funksjoner er første-klasses innbyggere i FP

- ▶ Funksjoner kan være **inndata**, f.eks.:

- ▶ `qs [] = []`

`qs (x:xs) = qs [y|y<-xs,y<=x] ++ [x] ++ qs [y|y<-xs,y>x]`

`qs :: Ord a => [a] -> [a]`

`gqs f [] = []`

`gqs f (x:xs) = qs [y|y<-xs, f y x] ++ [x] ++ qs [y|y<-xs, not(f y x)]`

`gqs :: (a -> a -> Bool) -> [a] -> [a]`

- ▶ `Data.List` har en funksjon

`sortBy :: (a->a->Ordering)->[a]->[a]`,

der `data Ordering = LT|GT|EQ ...`

f.eks. **lex** `a b = if a<b then LT else if a>b then GT else EQ`

lgh `a b = if length a < length b then LT`

`else if length a > length b then GT else EQ`

sortBy lex `xs` – sorterer `xs` etter `<` (lister i `xs`: leksikografisk)

sortBy lgh `xs` – vil sortere lister i `xs` etter lengde

Høyere-ordens funksjoner

- ▶ Funksjoner er første-klasses innbyggere i FP

- ▶ Funksjoner kan være **inndata**...

- ▶ ... og **utdata**

– f.eks., seksjoner:

$([2]++) :: \text{Num } a \Rightarrow [a] \rightarrow [a]$

$(++) :: [a] \rightarrow ([a] \rightarrow [a])$

$\text{zip } [1,2,3] :: \text{Num } t \Rightarrow [b] \rightarrow [(t, b)]$

$\text{zip} :: [a] \rightarrow [b] \rightarrow [(a, b)]$

- ▶ inn- og utdata (samtidig)

$f . g = \lambda x \rightarrow f (g x) \quad \neq \lambda x \rightarrow f g x$

$(.) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$

f.eks. $\text{odd } x = \text{not}(\text{even } x)$ kortere: $\text{odd} = \text{not} . \text{even}$

$\text{twice } f x = f (f x)$ kortere: $\text{twice } f = f . f$

..... $\text{twice} = \lambda f \rightarrow f . f$

Inn- og utdata: $tw\ f\ x = f\ (f\ x)$

$tw = \backslash f \rightarrow \backslash x \rightarrow f\ (f\ x)$

$tw :: (t \rightarrow t) \rightarrow t \rightarrow t$

Gitt $s = (1+)$ ($= \backslash x \rightarrow (1+) x$):

- $tw\ s = \backslash x \rightarrow s(s\ x)$
 $= s \circ s = (2+) :: Int \rightarrow Int$
- $tw\ (4+)\ 2 = 10$
 $tw\ (6+)\ 2 = 14$
- $tw\ (++\ "HaHa...\")\ "Hei!" = "Hei!\ HaHa...\ HaHa..."$
 $tw\ ("HaHa..." ++)\ "Hei!" = "HaHa...\ HaHa...\ Hei!"$
- $tw\ (3:)\ [5,6,7] = [3,3,5,6,7]$

Liste-funksjoner 'elementvis'

IP: `for (int i=0;i<a.length;++i) a[i] = ...`

FP: `iter [] = []`
`iter (h:t) = (... h) : iter t`

► Eksempler:

- `copy [] = []`
`copy (h:t) = h : copy t`
- `liad1 [] = []`
`liad1 (h:t) = h+1 : liad1 t`
- `lisq [] = []`
`lisq (h:t) = h*h : lisq t`

!! Kan programmeres med liste-komprehensjon

Funksjoner 'elementvis'

► Generelt:

```
copy [] = []  
copy (h:t) = h : copy t  
liad1 [] = []  
liad1 (h:t) = h+1 : liad1 t  
lisq [] = []  
lisq (h:t) = h*h : lisq t
```

```
map :: (a -> a) -> [a] -> [a]
```

```
map f [] = []
```

```
map f (h:t) = f h : map f t
```

f.eks. `map f [1,2,3]` gir `[(f 1), (f 2), (f 3)]`

► Nå:

```
copy = map id    :: [a] -> [a]
```

```
liad1 = map (\x -> x+1) :: Num a => [a] -> [a]
```

```
lisq = map (\x -> x*x)  :: Num a => [a] -> [a]
```

Flere eksempler på map

$\text{map} :: (a \rightarrow b) \rightarrow [a] \rightarrow [b]$

$\text{map } f [] = []$

$\text{map } f (h:xs) = f h : \text{map } f xs$

► Gitt:

► $\text{fst}(x,y) = x$

► $\text{snd}(x,y) = y$

► $\text{sq}(n) = n*n$

► $\text{map fst } [(1, 'en'), (2, 'to'), (3, 'tre')]$
 $\text{gir } [1,2,3]$

► $\text{map snd } [(1, 'en'), (2, 'to'), (3, 'tre')]$
 $\text{gir } ['en', 'to', 'tre']$

► $\text{concat } (\text{map snd } [(1, 'en'), (2, 'to'), (3, 'tre')])$
 $\text{gir } 'entotre'$

► $\text{map sq } (\text{map fst } [(1, 'en'), (2, 'to'), (3, 'tre')])$
 $\text{gir } [1,4,9]$

Eksempel: matriser

```
m = [ [(1,1),(1,2),(1,3),(1,4)],  
      [(2,1),(2,2),(2,3),(2,4)],  
      [(3,1),(3,2),(3,3),(3,4)] ]
```

Hva gir følgende?

▶ `head m` og `[head m]` `[(1,1),(1,2),(1,3),(1,4)]`

▶ `tail m`

▶ `map head m` `[(1,1),(2,1),(3,1)]`

▶ `map tail m`

Hvordan få n-te rad ? `row n m = m !! (n-1)`

... k-te kolonne ? `col k m = map (!! (k-1)) m`

transponert matrise: `trans m = [col i m | i <- [1..length (head m)]]`

Variasjoner

- ▶ `map f l` har samme lengde som `l`
- ▶ Hva hvis vi vil sile ut noen elementer? `p :: a -> Bool`
- ▶ `sil p [] = []`
`sil p (h:t) = if p h then h:sil p t else sil p t`
- ▶ Bedre i Haskell: `filter p xs = [x | x <- xs, p x]`
- ▶ `filter (\x -> x>0) [1,-1,0,2,-2]` gir `[1,2]`

f.eks: `strip a l = filter (/= a) l`

- ▶ `strip 1 [2,1,3,1,4,4,1,5]` gir `[2,3,4,4,5]`
- ▶ `strip '1' "21314415"` gir `"23445"`
- ▶ `strip "1" "21314415"` gir **feil type**
`strip :: Eq a => a -> [a] -> [a]`

foldr og foldl

- ▶ `sum_all [] = 0`
`sum_all (h:xs) = h + sum_all xs`
- ▶ `mul_all [] = 1`
`mul_all (h:xs) = h * mul_all xs`
- ▶ Bedre i Haskell: `sum`, `product`, ...!
- ▶ `foldr f v [] = v`
`foldr f v (h:xs) = f h (foldr f v xs)`
`sum_all = foldr (+) 0`
`mul_all = foldr (*) 1`

foldr

- ▶ `foldr f v [] = v`
`foldr f v (h:t) = f h (foldr f v t)`
`sum_all = foldr (+) 0`
`mul_all = foldr (*) 1`
- ▶ `foldr:: (e -> v -> v) -> v -> [e] -> v`
- ▶ `sum_all [1,2,3] = foldr (+) 0 [1,2,3] =`
`1 + (foldr (+) 0 [2,3]) =`
`1 + (2 + (foldr (+) 0 [3])) =`
`1 + (2 + (3 + (foldr (+) 0 []))) =`
`1 + (2 + (3 + 0))`
- ▶ `foldr` assosierer (sitt første argument) til høyre:
`foldr o v [x1, x2, x3, ..., xk] = x1 o (x2 o (x3 o (...(xk o v)...)))`

dvs., er ikke hale-rekursiv...

Bruk akkumulator

```
► sum_all 1 = sum_hale 0 1
  sum_hale v [] = v
  sum_hale v (h:t) = sum_hale (v+h) t
```

sh 0 [1,2,3] =

sh 0+1 [2,3] =

sh (0+1)+2 [3] =

sh ((0+1)+2)+3 [] = ((0+1)+2)+3

```
► foldl f v [] = v
  foldl f v (h:t) = foldl f (f v h) t
```

```
► sum_all = foldl (+) 0
```

```
► mul_all = foldl (*) 1
```

► foldl venstre-assosierer (sitt første argument)

$\text{foldl } \circ b [x_1, x_2, x_3, \dots, x_k] = (\dots(((b \circ x_1) \circ x_2) \circ x_3) \circ \dots) \circ x_k$

og er hale-rekursiv.

```
:t foldl :: (v -> e -> v) -> v -> [e] -> v
```

Hva gjør disse?

```
foldr f b [] = b  
foldr f b (h:t) = f h (foldr f b t)  
foldr o b [x1, x2, ..., xk] = x1 o (x2 o (... (xk o b) ...))
```

```
foldl f a [] = a  
foldl f a (h:t) = foldl f (f a h) t  
foldl o a [x1, x2, ..., xk] = (... ((a o x1) o x2) o ...) o xk
```

- ▶ $\text{foldr } (-) \ 0 \ [1,2,3,4] = 1 - (2 - (3 - (4 - 0))) = -2$
- ▶ $\text{foldl } (-) \ 0 \ [1,2,3,4] = (((0 - 1) - 2) - 3) - 4 = -10$
- ▶ $\text{foldl } (+) \ 0 \ [1,2,3,4] = 10 = \text{foldr } (+) \ 0 \ [1,2,3,4]$
- ▶ $\text{foldr } (\backslash_ \rightarrow (1+)) \ 0 \ [1,2,3,4] = \text{length}$
- ▶ $\text{foldr } (\backslash x \rightarrow \backslash xs \rightarrow xs++[x]) \ [] \ [1,2,3] =$
 $g \ 1 \ (\text{foldr } g \ [] \ [2,3]) = g \ 1 \ (g \ 2 \ (\text{foldr } g \ [] \ [3]))$
 $= g \ 1 \ (g \ 2 \ (g \ 3 \ (\text{foldr } g \ [] \ []))) = g \ 1 \ (g \ 2 \ (g \ 3 \ []))$
 $= g \ 1 \ (g \ 2 \ [3]) = g \ 1 \ [3,2] = [3,2,1] \dots\dots\dots = \text{reverse}$

Hva gjør disse?

```
foldr f b [] = b
foldr f b (h:t) = f h (foldr f b t)
foldr o b [x1, x2, ..., xk] = x1 o (x2 o (... (xk o b) ...))
```

```
foldl f a [] = a
```

```
foldl f a (h:t) = foldl f (f a h) t
```

```
foldl o a [x1, x2, ..., xk] = (...((a o x1) o x2) o ...) o xk
```

- ▶ `foldr (\x -> \xs -> xs++[x]) [] [1,2,3] = reverse`
- ▶ `foldl (\xs -> \x -> xs++[x]) [] [1,2,3] =`
`( ([] ++ [1]) ++ [2] ) ++ [3] = [1,2,3]`
- ▶ `foldl (\xs -> \x -> x:xs) [] [1,2,3] =`
`3 : 2 : 1 : [] ..... = reverse (hl.rek.)`

Et par eksempler til...

foldl $\circ$ b $[x_1, x_2, x_3, \dots, x_k] = (\dots((b \circ x_1) \circ x_2) \circ \dots) \circ x_k$

foldr $\circ$ b $[x_1, x_2, x_3, \dots, x_k] = x_1 \circ (x_2 \circ (\dots(x_k \circ b)\dots))$

sum = foldl (+) 0

= foldr (+) 0

product = foldl (*) 1

= foldr (*) 1

length = foldl (\ n _ -> n+1) 0

= foldr (\ _ n -> n+1) 0

or :: [Bool] -> Bool

= foldl (||) False = foldr (||) False

mer foldr...

`foldr f v [] = v`

`foldr f v (h:t) = f h (foldr f v t)`

`foldr o b [x1, x2, ..., xk] = x1 o (x2 o (... (xk o b) ...))`

`foldr :: (e -> [e] -> [e]) -> [e] -> [e] -> [e]`

`foldr (:) [] xs == xs`

- definer map ved å bruke foldr:

`map1 f li = foldr (\x -> \xs -> (f x):xs) [] li`

`map2 f = foldr (\x -> (f x:) ) []`

`map3 f = foldr ((:) . f) []`

- og filter....

`filter p l = foldr (\x -> \xs -> if p x then x:xs else xs) [] l`

`filter1 p = foldr (\x -> \xs -> if p x then x:xs else xs) []`

`filter2 p = foldr (\x -> if p x then (x:) . id else id) []`

`filter2 p = foldr (\x -> if p x then (x:) else id) []`

- ? Hvorfor ikke `foldl :: ([e] -> e -> [e]) -> ...` og venstre-assosierer.

`foldl (:) [] [x1, x2, x3, ..., xk] = (...(([] : x1) : x2)...) : xk`

`foldl (\ xs x -> x:xs) [] li = reverse`

HO liste-rekursjon, oppsummering

- ▶ `map :: (a -> b) -> [a] -> [b]`
- ▶ Høyre rekursjon: `[] ---> v`, `(h:t) ---> (f h ...)`, hvor ... står for et rekursivt kall

`foldr f v [] = v`

`foldr f v (h:t) = f h (foldr f v t)`

`:t foldr :: (e -> v -> v) -> v -> [e] -> v`

- ▶ Venstre rekursjon: `[] ---> v`, `(h:t) ---> ... (f v h)`; akkumulator, hale-rekursjon

`foldl f v [] = v`

`foldl f v (h:t) = foldl f (f v h) t`

`:t foldl :: (v -> e -> v) -> v -> [e] -> v`

- ▶ `foldr` og `foldl` "speilbilder" hverandre:

$\text{foldr} \circ b [x_1, x_2, x_3, \dots, x_k] = x_1 \circ (x_2 \circ (x_3 \circ (\dots (x_k \circ b) \dots)))$

$\text{foldl} \circ b [x_1, x_2, x_3, \dots, x_k] = (\dots (((b \circ x_1) \circ x_2) \circ x_3) \circ \dots) \circ x_k$

Funksjonskomposisjon

`foldr fu v [] = v`

`foldr fu v (h:t) = fu h (foldr fu v t)`

► `compose = foldr (.) id` $(.) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$

► `compose [f,g,h] = foldr (.) id [f,g,h]`
 $= (.) f (foldr (.) id [g,h])$
 $= (.) f ((.) g (foldr (.) id [h]))$
 $= (.) f ((.) g ((.) h (foldr (.) id [])))$
 $= (.) f ((.) g ((.) h id)) = (.) f ((.) g h)$
 $= (.) f (g . h) = f . (g . h) = \lambda x \rightarrow f (g (h x))$

► `foldr . id [f1, f2, f3, ..., fk] = f1.(f2.(f3.(...(fk.id)...)))`

► `:t compose :: [t -> t] -> t -> t`

► funksjonskomposisjon assosierer til høyre

dvs. $(f . g . h) x = f (g (h(x)))$

men siden den er assosiativ, er dette uvesentlig for resultatet, dvs.

$\forall x : (f_1.(f_2.f_3))x = ((f_1.f_2).f_3)x$

altså `compose = foldl (.) id`

Fuksjonskomposisjon . – for eksempel

- ▶ for å lage anonyme funksjoner (uten lambda)
vanligvis, som aktuelle argumenter til andre funksjoner
- ▶ for en liste med tall, gjør alle til negative
 $[1, 3, -6, -7, 8] \rightarrow [-1, -3, -6, -7, 8]$

```
f1 = map (\ x -> negate (abs x))
```

```
f2 = map (negate . abs)
```

- ▶ ta absolutt verdi av ethvert tall og kvadratroten av det +1

```
g1 = map ( x -> sqrt ((abs x)+1))
```

```
g1 = map (sqrt . (+1) . abs)
```

Husk! . assosierer til høyre: $(f . g . h) x = f (g (h (x)))$

```
f x = ceiling . negate . tan . cos (max 30 x)
```

```
f = ceiling . negate . tan . cos . (max 30)
```

NB! Funksjonsapplikasjon assosierer til venstre!

► $a\ b\ c\ x = ((a\ b)\ c)\ x$

$a :: tb \rightarrow (tc \rightarrow (tx \rightarrow tres))$

'curried' funksjoner tar bare ett argument

► $(+)\ (+2)\ 1\ 7$

Non type-variable argument in the constraint: (Num (a -> a))...

fordi $((+)\ (+2))\ 1\ 7$ blir feiltypet

og initiell evaluering av $((+)\ (+2))$ gir

Non type-variable argument in the constraint: (Num (a -> a))...

► $(+)\ ((+2)\ 1)\ 7$

$=\ (+)\ ((+2)\ 1)\ 7$

$=\ (+)\ (3)\ 7$

$=\ (+3)\ 7$

$=\ 10$

“Strikt” funksjonsapplikasjon – \$

- ▶ lar oss skrive enda færre paranteser

```
f = sum ( filter (>10) (map (*2) [1..10]) )
```

- ▶ `exp1 $ exp2` – tvinger bruk av `exp2` som parameter til `exp1`

```
f = sum $ filter (>10) (map (*2) [1..10])
```

- ▶ \$ associerer til højre, dvs. $f \$ g \$ h \times = f \$ (g \$ (h \times))$

```
f = sum $ filter (>10) $ map (*2) [1..10]
```

§ muliggjør også bruk av en verdi som om den var en funksjon

(§) :: $(a \rightarrow b) \rightarrow a \rightarrow b$, så

(§ 9) :: Num a => (a -> b) -> b, er en funksjon

2 + \$ 9 blir feil, siden \$ 9 forventer en funksjon, men
 (+2) \$ 9 == 11 også (\$ 9) (+3) == 12.

- ▶ snittet (\$ 9) kan sendes som funksjonsargument, f.eks.,

```
map ($ 9) [(+2), sqrt] = [11.0, 3.0]
```

repeat :: a -> [a]

► repeat x = [x, x, x, x, x, ...]

– uendelig liste?
repeat = iterate id

kan brukes takket være “lat” evaluering

f.eks. fill inn en liste ls med en verdi x(=0) tom. n(=8) posisjoner

fill ls x n = take n (ls ++ repeat x)

fill [1,2,3,4] 0 8 = [1,2,3,4,0,0,0,0]

?? fill inn listen i fronten med x til 8 posisjoner

fillf [1,2,3,4] 0 8 = [0,0,0,0,1,2,3,4]

fillf ls x n = take n (repeat x ++ ls)

fillf ls x n = reverse (take n (reverse ls ++ repeat x))

iterate :: (a -> a) -> a -> [a]

► $\text{iterate } f \ x = [x, f \ x, f \ (f \ x), f \ (f \ (f \ x)), \dots]$

iterate (1+) 1 ? kan brukes takket være “lat” evaluering

f.eks. binært tall skrevet som en liste: [1, 0, 1, 1]
skal omregnes til desimaltall $= 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0$

► $\text{iterate } (2^*) \ 1 = [1, 2, 2^2, 2^3, 2^4, \dots]$

► $\text{bin2dec } bt = \text{sum } [w * b \mid (w, b) <- \text{zip } \text{exp2 } (\text{reverse } bt)]$
where $\text{exp2} = \text{iterate } (2^*) \ 1$

foldl f a [] = a foldl :: (v -> e -> v) -> v -> [e] -> v

foldl f a (h:t) = foldl f (f a h) t

► $b2d = \text{foldl } (\backslash a \rightarrow \backslash h \rightarrow (a * 2) + h) \ 0$

$\text{fl } f \ 0 \ [1, 0, 1] = \text{fl } f \ (0 * 2 + 1) \ [0, 1] = \text{fl } f \ 1 \ [0, 1] =$
 $= \text{fl } f \ (1 * 2 + 0) \ [1] = \text{fl } f \ 2 \ [1] = \text{fl } f \ (2 * 2 + 1) \ []$
 $((0 * 2 + 1) * 2 + 0) * 2 + 1 = 5$

Inneholder en liste tallet 0?

- ▶ `zero = or . map (==0)`
- > `zero (repeat 0) == True`

(r) `zr :: [Int] -> Bool`
`zr = foldr ( (||).(==0) ) False`
> `zr (repeat 0) == True`

(l) `zl :: [Int] -> Bool`
`zl = foldl ( flip ((||).(==0)) ) False`
> `zl (repeat 0) ....`

(r) `foldr f v [] = v`
`foldr f v (h:ts) = f h (foldr f v ts)`
 $x_1 \circ (x_2 \circ (x_3 \circ (\dots (x_k \circ v) \dots)))$

(l) `foldl f v [] = v`
`foldl f v (h:ts) = foldl f (f v h) ts`
 $(\dots (((v \circ x_1) \circ x_2) \circ x_3) \circ \dots) \circ x_k$
`foldl f v [x1, x2, x3...] = foldl f (f v x1) [x2, x3...] =`
`foldl f (f (f v x1) x2) [x3...] = foldl f (f(f(f v x1) x2) x3) [...]`